LAB 02 Dart foundations and Flutter project anatomy

Unit 1 • 3-hour guided laboratory • CLO-5

Learning goal. Apply Dart types, null safety, collections and reusable functions inside a Flutter interface and recognize the purpose of common project files.

Core concepts. Dart var/final/const; types and null safety; lists, map, where; functions; pubspec.yaml; lib/, android/, test/, assets; hot reload.

Before you start. Lab 01 toolchain and a recognized Android phone.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab02
cd mad_lab02
flutter run
```

Anatomy of a generated project

A Flutter Android application has one PROJECT ROOT: the folder containing pubspec.yaml. In Android Studio, use the Project or Project Files view (not the filtered Android view) to study the complete layout below. The assets/ directory is student-created; generated directories and exact native filenames may vary by Flutter version.

ILLUSTRATED DIRECTORY TREE | Example for flutter create --platforms=android mad_lab02

```
mad_lab02/                          <- PROJECT ROOT
|-- lib/
|   `-- main.dart                  <- start editing Dart UI here
|-- test/
|   `-- widget_test.dart           <- generated starter test
|-- android/
|   |-- app/
|   |   |-- src/main/
|   |   |   |-- AndroidManifest.xml
|   |   |   `-- kotlin/.../MainActivity.kt  <- native bridge
|   |   `-- build.gradle.kts        <- app Android configuration
|   |-- settings.gradle.kts        <- generated Gradle settings
|   `-- gradle.properties
|-- assets/                         <- create yourself, if needed
|   `-- notes.txt
|-- pubspec.yaml                    <- package metadata + assets
|-- pubspec.lock                    <- resolved dependencies
|-- analysis_options.yaml          <- Dart analysis rules
|-- .gitignore                      <- generated exclusions
|-- README.md                       <- project setup notes
|-- .dart_tool/                     <- generated; do not commit
`-- build/                          <- generated; do not commit
```

Read the tree from top to bottom: your Dart screen code belongs in lib/, Android-specific settings and native code belong in android/, automated tests belong in test/, and dependencies/assets are declared in pubspec.yaml. The tree omits several Gradle wrapper/support files to stay readable. A regular multi-platform project may also contain ios/, web/, macos/, linux/ and windows/; this course uses Android-only generation.

Path / file	Purpose and edit policy
pubspec.yaml	Single package manifest; configure dependencies, SDK constraints, version, and bundled assets. EDIT deliberately; YAML indentation matters.
pubspec.lock	Records resolved package versions for repeatable builds; for application projects normally keep it in source control.
lib/main.dart	Dart entry point; main() calls runApp() to attach the root widget. EDIT for each solved task.
lib/screens/, lib/widgets/	Optional folders STUDENTS create as an app grows; screens group pages and widgets hold reusable UI.
test/widget_test.dart	Flutter widget tests. Replace the generated starter test when your main.dart no longer contains its starter MyApp.

Path / file	Purpose and edit policy
android/app/src/main/AndroidManifest.xml	Android application label, activities and requested Android permissions; change permissions only when a feature requires them.
android/app/build.gradle.kts	Android application ID, SDK compatibility and build configuration. Some older templates use build.gradle (Groovy).
android/app/src/main/kotlin/.../MainActivity.kt	Android entry activity and optional platform-channel Kotlin code. Actual package directories match the generated organization ID.
android/settings.gradle.kts and Gradle wrapper	Android build infrastructure generated by Flutter; normally leave unchanged for introductory labs.
assets/	You create this directory for images, fonts or JSON; declare each asset (or appropriate directory) under flutter: in pubspec.yaml.
analysis_options.yaml	Static-analysis lint rules for Dart; flutter analyze uses these rules.
.gitignore	Excludes temporary output, local IDE state and secrets; preserve and extend the generated file rather than overwriting.
build/ and .dart_tool/	Generated build output and tool caches; do not modify or commit to Git.
README.md	Plain-language project purpose, prerequisites, setup commands, screenshots and contribution link.

How a Flutter application starts and displays a screen

EXECUTION FLOW | What happens when you press Run

```
Android launches the Flutter Activity
        |
Dart main() executes in lib/main.dart
        |
runApp(const RootWidget()) installs the widget tree
        |
MaterialApp selects theme, routes and home widget
        |
Scaffold arranges AppBar, body and floating actions
        |
Widgets are built, laid out and painted on the phone
```

The tree is declarative: widgets describe the interface at a moment in time, while Flutter decides how to lay out and repaint it. setState marks relevant State objects dirty and schedules new widget builds; it is not a direct instruction to paint pixels. Native MainActivity hosts the Flutter engine on Android but typically contains no Dart UI code.

Open, edit and run the correct files

WORKFLOW 1 | Open the project root

Choose File > Open and select mad_lab02 itself (the folder containing pubspec.yaml). Wait for package resolution. The preferred editor view for studying folders is Project / Project Files; the Android view is better for native Gradle inspection.

WORKFLOW 2 | Replace only the requested source file

When a solved activity is labelled lib/main.dart, select that file and replace its ENTIRE contents with the provided complete example. Do not paste the example into AndroidManifest.xml, MainActivity.kt or the build script. Save before running.

WORKFLOW 3 | Install packages only when instructed

flutter pub get resolves packages already declared in pubspec.yaml; flutter pub add PACKAGE_NAME adds a new dependency and fetches it. Most early labs use only the Flutter SDK and need no extra packages. Do not edit pubspec.lock manually.

REPRODUCIBLE VALIDATION | Use the project root; run commands in order

```
flutter pub get
dart format lib test
flutter analyze
flutter devices
flutter run
```

After the code: pub get downloads dependencies, dart format applies Dart formatting, analyze flags static errors, devices confirms the phone target, and run compiles, installs and starts the app. These checks should be executed inside the directory containing pubspec.yaml. If your generated starter widget test still references the old app after you replace main.dart, first update that test to match the new class before running flutter test.

How pubspec.yaml controls packages and assets

This YAML file is configuration, not Dart code. The dependencies section lists runtime packages; dev_dependencies lists tools such as flutter_test and linters. The flutter: section configures Flutter-specific assets and fonts. Keep indentation with spaces; tabs can break YAML. The small example below is an EDIT TO THE EXISTING flutter: section, not a complete replacement pubspec.yaml.

ADD A LOCAL TEXT ASSET | Git Bash / Linux / macOS terminal

```
mkdir -p assets
printf 'CSC303: bundled local data\n' > assets/notes.txt
```

EDIT EXISTING pubspec.yaml | Insert assets: under the existing flutter: key

```
flutter:
  uses-material-design: true
  assets:
    - assets/notes.txt
```

Explanation: create the file BEFORE declaring it; the asset path is relative to pubspec.yaml. Do not add a SECOND flutter: block; insert assets: into the block Flutter generated, then save and run flutter pub get. If the example app does not load this asset yet, its inclusion changes the bundle but not the screen. Official guidance: https://docs.flutter.dev/ui/assets/assets-and-images.

AndroidManifest.xml, Gradle and generated Kotlin code

AndroidManifest.xml is the native Android declaration of app components and permissions. For an HTTP/HTTPS API feature, release builds commonly need the INTERNET permission. Put the one-line declaration directly inside the existing manifest root ONLY when networking is needed; never replace the entire manifest with this line.

ANDROID PERMISSION FRAGMENT | Add inside the existing <manifest> element if needed

```
<uses-permission android:name="android.permission.INTERNET" />
```

The Android app Gradle file (usually android/app/build.gradle.kts in modern templates) controls namespace, applicationId, SDK levels and release signing. applicationId identifies the installed app; changing it can affect Firebase registrations or updates. compileSdk is the Android API used to compile, minSdk is the oldest supported Android version, and targetSdk communicates intended Android behavior. Flutter templates supply compatible defaults: edit only when a package, device policy or deployment requirement actually calls for it.

MainActivity.kt is a native Android entry point, normally a short class that extends FlutterActivity. Dart UI changes belong in lib/, not MainActivity. Lab 14 later adds a method channel as an intentional exception. Do not commit android/local.properties (local SDK paths), generated build output, or signing credentials.

Hot reload, hot restart and debugging a real phone

DEBUG 1 | Make a Dart-only UI change

While flutter run is active, modify a Text label in lib/main.dart and save. Press r in the terminal or use Android Studio Hot Reload. Hot reload usually preserves the current widget state, so counters and form inputs often remain unchanged.

DEBUG 2 | Know when to restart

Press R in the flutter run terminal for Hot Restart: the Dart app is restarted and local widget state is reset. If you change AndroidManifest.xml, native Kotlin, Gradle, permissions, plugins or native resources, stop the process and start a new build; hot reload cannot apply native Android code changes.

DEBUG 3 | Debug logically rather than guessing

Click in the left gutter beside a Dart line to set a breakpoint, start Debug, trigger the event on the phone, then inspect locals and the call stack. Use Run/Debug output for Dart errors, the Flutter Inspector for widget-tree/layout problems, and Android Logcat for native crashes. A yellow-black overflow warning is a layout problem, not a compiler error.

FINAL LAB 02 CHECKS | Run only with a widget test matching the CURRENT app

```
dart format lib test
flutter analyze
flutter test
flutter build apk --debug
```

IMPORTANT: flutter create includes a starter widget test tied to the generated starter app. Replacing lib/main.dart for either solved task invalidates that test unless you rewrite it. Use the working example in Lab 01 as a test replacement pattern. The debug APK is written under build/app/outputs/flutter-apk/. Official guidance: https://docs.flutter.dev/tools/hot-reload and https://docs.flutter.dev/tools/android-studio.

Solved task 1 | Reusable Dart function: grade analyzer

Problem and objective. Create a grading widget that converts assignment, midterm and final raw marks into a 100-point score and displays a grade. This is a PROGRAMMING exercise and does not replace the official course grading policy.

lib/main.dart

```
import 'package:flutter/material.dart';

String resultFor(double a, double m, double f) {
  final score = a + m + f;
  if (score >= 85) return 'A';
  if (score >= 70) return 'B';
  if (score >= 60) return 'C';
  if (score >= 50) return 'D';
  return 'F';
}

void main() => runApp(const MaterialApp(home: GradeDemo()));

class GradeDemo extends StatefulWidget {
  const GradeDemo({super.key});
  @override
  State<GradeDemo> createState() => _GradeDemoState();
}

class _GradeDemoState extends State<GradeDemo> {
  double assignment = 20, midterm = 18, finalExam = 42;

  @override
  Widget build(BuildContext context) {
    final score = assignment + midterm + finalExam;
    return Scaffold(
      appBar: AppBar(title: const Text('Dart grade analyzer')),
      body: ListView(padding: const EdgeInsets.all(18), children: [
        for (final item in [
          ('Assignment /25', assignment, 25.0),
          ('Midterm /25', midterm, 25.0),
          ('Final /50', finalExam, 50.0),
        ])
          Column(children: [
            Text('${item.$1}: ${item.$2.toStringAsFixed(0)}'),
            Slider(
              value: item.$2, max: item.$3,
              onChanged: (v) => setState(() {
                if (item.$1.startsWith('Assignment')) assignment = v;
                if (item.$1.startsWith('Midterm')) midterm = v;
                if (item.$1.startsWith('Final')) finalExam = v;
              }),
            ),
          ]),
        Card(child: Padding(
          padding: const EdgeInsets.all(20),
          child: Text('Total: ${score.toStringAsFixed(0)}/100  '
              'Grade: ${resultFor(assignment, midterm, finalExam)}'),
        )),
      ]),
```

```
    );
  }
}
```

How the solution works. The three numeric state fields store user inputs. A pure function resultFor maps the sum into a grade category. Record fields item.$1, item.$2 and item.$3 demonstrate positional records in modern Dart; Slider clamps input to its declared maximum.

Expected output. Moving sliders updates totals and the grade immediately. The example starts at 80/100 (B).

Check your work. Try 25+25+50 and 0+0+0, then explain why the result function is simpler to test than calculations embedded in build().

Solved task 2 | Lists, maps and null safety in a directory

Problem and objective. Use in-memory structured data, filtering and optional values in a student directory. The example does not use a server.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: DirectoryPage()));

class Student {
  final String name;
  final String department;
  final String? phone;
  const Student(this.name, this.department, this.phone);
}

class DirectoryPage extends StatefulWidget {
  const DirectoryPage({super.key});
  @override
  State<DirectoryPage> createState() => _DirectoryPageState();
}

class _DirectoryPageState extends State<DirectoryPage> {
  final entries = const [
    Student('Ayesha', 'CS', '0300-0000001'),
    Student('Hamza', 'SE', null),
    Student('Hina', 'CS', '0300-0000003'),
    Student('Bilal', 'SE', null),
  ];
  String selected = 'All';

  @override
  Widget build(BuildContext context) {
    final visible = entries.where((s) =>
        selected == 'All' || s.department == selected).toList();
    return Scaffold(
      appBar: AppBar(title: const Text('Student directory')),
      body: Column(children: [
        SegmentedButton<String>(
          segments: const [
            ButtonSegment(value: 'All', label: Text('All')),
            ButtonSegment(value: 'CS', label: Text('CS')),
            ButtonSegment(value: 'SE', label: Text('SE')),
          ],
          selected: {selected},
          onSelectionChanged: (v) => setState(() => selected = v.first),
        ),
        Expanded(child: ListView.builder(
          itemCount: visible.length,
          itemBuilder: (context, i) {
            final s = visible[i];
            return ListTile(
              leading: CircleAvatar(child: Text(s.name[0])),
              title: Text(s.name),
              subtitle: Text('${s.department} • ${s.phone ?? 'No phone'}'),
            );
```

```
          },
        )),
      ]),
    );
  }
}
```

How the solution works. Student is an immutable data model. Its phone is nullable; the ?? operator supplies safe fallback text. List.where returns the selected subset and ListView.builder renders only needed list children.

Expected output. All displays four people; CS and SE each display two. Missing numbers show No phone.

Check your work. Add one entry with no phone, change the selected filter and confirm no null exception occurs.

Scenario-based practice — unsolved

Scenario 1. Merit calculation. A scholarship form must compare GPAs and reject invalid negative or greater-than-4 values. Specify types, validation rules, test boundaries and use a pure function.

Scenario 2. Class roster. The CS office must group records by semester and search partial names while gracefully handling missing emails. Design a Student model and user-facing error states.

Technical viva — justify your reasoning

1. Why can const objects improve allocations and rebuild cost?
2. When do final and const differ?
3. Why is a nullable String? not assignable directly to String?
4. What does List.where return before toList() is called?
5. Why should grading thresholds be tested at exact boundaries?
6. Which changes require hot restart instead of hot reload?
7. Why can an Android release app fail to call an API even when the debug app connects successfully? Which file would you inspect?
8. Distinguish pubspec.yaml from pubspec.lock, and explain which files should be committed for an application project.

Submission and self-check

Both screenshots, flutter analyze output, commit SHA and a short explanation of null safety and project structure.